

Job Market Analytics Dashboard

Steps 1 & 2 — Project Setup + Adzuna API Connection

Project Overview

A full-stack web application that pulls live job postings from the Adzuna API, cleans and stores them in Supabase, and presents an interactive analytics dashboard showing market trends — most in-demand skills, top locations, salary ranges, and more. Built with Flask + Pandas + Plotly. Deployed on Railway.

Stack Layer	Technology
Web Framework	Flask
Data Source	Adzuna API (free, official JSON)
Data Processing	Pandas
Database	Supabase (Postgres)
Charts	Plotly
ML (stretch)	scikit-learn
Deployment	Railway (auto-deploy from GitHub)

STEP 1 — Project Setup

Status: ✓ Complete

Created the full project folder structure, virtual environment, installed initial packages, and confirmed the Flask development server runs locally.

1.1 Create Project Folder

Location: C:\Users\Jabir Raza\Desktop\All Projects\job-market-dashboard

Open in VS Code: File → Open Folder → select job-market-dashboard

1.2 Create Virtual Environment

Run in terminal:

```
python -m venv venv
```

Activate it:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned  
venv\Scripts\activate
```

You should see (venv) appear at the start of your terminal prompt.

ERROR ENCOUNTERED & FIX

**ERROR: PowerShell
blocked venv activation**

venv\Scripts\activate cannot be loaded
because running scripts is disabled on this
system.

**FIX: Run: Set-ExecutionPolicy -Scope
Process -ExecutionPolicy RemoteSigned
then activate again.**

1.3 Install Packages

```
pip install flask gunicorn python-dotenv requests beautifulsoup4  
pip freeze > requirements.txt
```

Note: pip also installs Flask's dependencies automatically — Jinja2, Werkzeug, Click, itsdangerous, MarkupSafe. These appear in requirements.txt even though you never typed them. This is normal — they are Flask's internal requirements.

1.4 File Structure Created

File / Folder	Purpose
app.py	Flask entry point. All routes defined here.
requirements.txt	All Python dependencies — Railway installs from this on every deploy.
runtime.txt	Contains: python-3.11.9 — tells Railway which Python to use.
Procfile	Contains: web: gunicorn app:app — tells Railway how to start the app.
.env	Stores secret API keys locally. Never pushed to GitHub (gitignored).
.gitignore	Tells git to ignore: venv/ .env __pycache__ *.pyc
scraper/	Folder — will hold fetch_jobs.py and clean_data.py
static/css/style.css	CSS file — dark theme matching portfolio. Empty for now.
templates/dashboard.html	Main dashboard page. Shows "Coming soon..." for now.

Why each config file exists:

File	What it does	Without it
requirements.txt	Lists all libraries so Railway can install them	Railway has no idea what to install → crash
runtime.txt	Pins Python to 3.11.9	Railway tries Python 3.13 → build fails (we learned this from portfolio)
Procfile	Tells Railway: run gunicorn app:app	Railway doesn't know how to start the app
.env	Stores API keys for local dev	os.getenv() returns None → all API calls fail
.gitignore	Prevents venv/ and .env from being pushed	Secrets go public + repo becomes 100MB+

1.5 Minimal app.py

```

from flask import Flask, render_template
from dotenv import load_dotenv
import os
load_dotenv()
app = Flask(__name__)
@app.route("/")
def dashboard():
    return render_template("dashboard.html")
if __name__ == "__main__":
    app.run(debug=True)

```

1.6 Test Locally

```
python app.py
```

Result: ✓ Running at <http://127.0.0.1:5000> — "Job Market Analytics Dashboard" visible in browser.

STEP 2 — Adzuna API Connection

Status: ✓ Complete

Registered for a free Adzuna developer account, obtained App ID and API Key, added them to .env, and wrote fetch_jobs.py which successfully pulled 20 live job postings from the Adzuna API.

2.1 Why Adzuna API?

Option	Pros	Cons
Adzuna API	Free, official, clean JSON, no scraping risk, salary data included	Unfocused, limited free tier
Web scraping	Any site	Legal risk, breaks when site redesigns, messy HTML
RemoteOK API	Free, no auth needed	Remote-only jobs, limited data fields

Kaggle CSV	Clean data	Static — not live, defeats the dashboard purpose
------------	------------	--

2.2 Register at developer.adzuna.com

Field	Value used
Username	Shakir Raza
Email	shakirarain9@gmail.com
Organisation	Shakir Raza
Application type	Personal or academic research
Average Monthly Visitors	0-5000
Primary Market	Other
Primary Industry	Technology

2.3 Credentials Obtained

Key	Value	Where used
Application ID (App ID)	87928b97	Passed in every API request as app_id parameter
Application Key (API Key)	5f8132d6176bc5b4253722cca6c910cb	Passed as app_key parameter

Added to .env file:

```
ADZUNA_APP_ID=87928b97
ADZUNA_API_KEY=5f8132d6176bc5b4253722cca6c910cb
```

2.4 scraper/fetch_jobs.py

Full code:

```
import requests
import os
from dotenv import load_dotenv
load_dotenv()
APP_ID = os.getenv("ADZUNA_APP_ID")
API_KEY = os.getenv("ADZUNA_API_KEY")
def fetch_jobs(what="python developer", where="london", results_per_page=20, page=1):
    url = f"https://api.adzuna.com/v1/api/jobs/gb/search/{page}"
    params = {
        "app_id": APP_ID,
        "app_key": API_KEY,
        "results_per_page": results_per_page,
        "what": what,
        "where": where,
        "content-type": "application/json",
    }
    response = requests.get(url, params=params)
    if response.status_code == 200:
        data = response.json()
        jobs = data.get("results", [])
        print(f"Fetched {len(jobs)} jobs for '{what}' in '{where}'")
        return jobs
    else:
        print(f"Error: {response.status_code}")
        return []
if __name__ == "__main__":
    jobs = fetch_jobs()
    for job in jobs[:3]:
```

```

print("Title: ", job.get("title"))
print("Company: ", job.get("company", {}).get("display_name"))
print("Location:", job.get("location", {}).get("display_name"))
print("Salary: ", job.get("salary_min"), "-", job.get("salary_max"))
print("URL: ", job.get("redirect_url"))

```

2.5 How the API URL Works

Part	What it means
https://api.adzuna.com/v1/api/jobs/	Base URL for all Adzuna job API calls
gb/	Country code — gb = United Kingdom (has most data on free tier)
search/1	Search endpoint, page 1 of results
?app_id=...	Your Application ID — identifies your app
&app_key=...	Your API Key — authenticates the request
&what=python developer	Job title / keyword to search for
&where=london	Location to search in
&results_per_page=20	How many jobs to return (max 50 on free tier)

2.6 Test Result

Command run:

```
python scraper/fetch_jobs.py
```

Output received:

#	Title	Company	Location	Salary Range
1	Backend Python Developer	XACT PLACEMENTS LIMITED	London, UK	60,000 - 80,000
2	Python Developer - mthree	mthree	London, UK	66,663 - 66,663
3	Senior Django / Python Developer	Sanderson	London, UK	156,000 - 182,000

Result: ✓ 20 jobs fetched successfully with all fields populated.

2.7 What the Raw JSON Looks Like

Each job returned by the API is a Python dict with these key fields:

JSON Field	What it contains	Example value
title	Job title string	Backend Python Developer
company.display_name	Company name	XACT PLACEMENTS LIMITED
location.display_name	Location string	London, UK
salary_min	Minimum salary (float or None)	60000.0
salary_max	Maximum salary (float or None)	80000.0
description	Full job description text	We are looking for a Python...
redirect_url	Link to original job posting	https://www.adzuna.co.uk/...
created	ISO timestamp when posted	2026-06-10T12:34:56Z
category.label	Job category	IT Jobs

Build Progress

Step	Description	Status
------	-------------	--------

1	Project setup — venv, packages, file structure, local Flask server	✓ Done
2	Adzuna API — register, get keys, write fetch_jobs.py, pull live data	✓ Done
3	Data cleaning — Pandas, normalize fields, extract skill tags	Next →
4	Supabase — create jobs table, store first batch	
5	Dashboard route — query Supabase, pass data to templates	
6	Plotly charts — skills, location, salary charts	
7	Jobs listing page — /jobs route with filters	
8	UI polish — dark CSS theme	
9	GitHub + Railway deployment	
10	Add to portfolio via /admin	
11	(Stretch) ML salary prediction model	